Java Stream API - Custom Class Interview Questions (Full Code)

```
package streamapi_custom_class;

import java.util.*;
import java.util.stream.Collectors;

public class StreamApiCustomClassInterviewQuestion {
    record Employee(Long id,String name,int age ,String department,double salary){}
    public static void main(String[] args) {
        List<Employee> employees = List.of(
                new Employee(1L, "Amit Sharma", 28, "IT", 65000),
                new Employee(2L, "Priya Singh", 30, "HR", 55000),
                new Employee(3L, "Rahul Verma", 26, "Finance", 60000),
                new Employee(4L, "Sneha Gupta", 32, "IT", 72000),
                new Employee(5L, "Vikas Kumar", 29, "Sales", 50000),
                new Employee(6L, "Neha Agarwal", 27, "Marketing", 58000),
                new Employee(7L, "Rohit Mishra", 35, "Finance", 75000),
                new Employee(8L, "Pooja Yadav", 31, "HR", 62000),
                new Employee(9L, "Ankit Jain", 24, "IT", 54000),
                new Employee(10L, "Kavita Singh", 33, "Sales", 67000),
                new Employee(11L, "Manish Tiwari", 36, "IT", 82000),
                new Employee(12L, "Deepak Pandey", 28, "Marketing", 56000),
                new Employee(13L, "Ritu Saxena", 34, "Finance", 71000),
                new Employee(14L, "Saurabh Srivastava", 29, "IT", 63000),
                new Employee(15L, "Meena Patel", 27, "HR", 52000),
                new Employee(16L, "Arjun Reddy", 31, "Sales", 69000),
                new Employee(17L, "Nisha Kapoor", 26, "Marketing", 57000),
                new Employee(18L, "Aditya Chauhan", 38, "Finance", 88000),
                new Employee(19L, "Simran Kaur", 25, "IT", 53000),
                new Employee(20L, "Tarun Malhotra", 40, "Management", 95000),
                new Employee(20L, "Tarun Malhotra", 40, "Management", 95000)
        );

        // Find the maximum salary among all employees
        var max = employees.stream().mapToDouble(Employee::salary).max().getAsDouble();
        System.out.println("max salary:"+max);

        // Find the employee who has the maximum salary
        Employee maxSalaryEmployee = employees.stream().max(Comparator.comparing(Employee::salary)).get(
        System.out.println("maxSalaryEmployee:"+maxSalaryEmployee);

        // Find the second highest salary
        double secondMaxSalary = employees.stream()
                .sorted(Comparator.comparing(Employee::salary).reversed())
                .skip(1)
                .findFirst()
                .get()
                .salary();
        System.out.println("secondMaxSalary:"+secondMaxSalary);

        // Find the employee with the second highest salary
        Employee secondMaxSalaryEmployee = employees.stream()
                .sorted(Comparator.comparing(Employee::salary).reversed())
                .skip(1)
                .findFirst()
                .get();

        System.out.println("secondMaxSalaryEmployee:"+secondMaxSalaryEmployee);

        // Find department-wise maximum salary
        employees.stream()
                .collect(Collectors.groupingBy(Employee::department))
                .entrySet()
                .stream()
                .collect(Collectors.toMap(
                        e->e.getKey(),
```

```
                        e->e.getValue().stream()
                                .max(Comparator.comparing(Employee::salary)).get().salary()
                )).forEach((k,v)-> System.out.println(k+":"+v));

        // Group employees by department
        Map<String, List<Employee>> map = employees.stream()
                .collect(Collectors.groupingBy(Employee::department));

        // Find department-wise second highest salary
        Map<String, Double> depaermentWiseSecondMaximum = map.entrySet()
                .stream()
                .collect(Collectors.toMap(e -> e.getKey(),
                        e -> e.getValue().stream()
                                .sorted(Comparator.comparing(Employee::salary).reversed())
                                .skip(1)
                                .findFirst()
                                .get()
                                .salary()
                ));

        System.out.println(depaermentWiseSecondMaximum);

        // Double salary for employees with age > 30
        employees.stream()
                .filter(e->e.age()>30)
                .map(e->new Employee(e.id(),e.name(),e.age(),e.department(),e.salary()*2))
                .toList()
                .forEach(System.out::println);

        // Sort employees by name
        employees.stream()
                .sorted(Comparator.comparing(Employee::name))
                .toList()
                .forEach(System.out::println);

        // Sort employees by name and age
        employees.stream()
                .sorted(Comparator.comparing(Employee::name).thenComparing(Employee::age))
                .toList()
                .forEach(System.out::println);

        // Count employees in each department
        employees.stream()
                .collect(Collectors.groupingBy(Employee::department))
                .entrySet()
                .stream()
                .collect(Collectors.toMap(e->e.getKey(),e->e.getValue().stream().count()))
                .forEach((k,v)-> System.out.println(k+":"+v));
    }
}
```